

A Vision-to-Music System for Accessible Interactive Sound Experiences Using Computer Vision and Csound

Xuanzhi (Will) Chen and Richard Boulanger

Berklee College of Music, Boston, MA, USA
willchen5258@gmail.com rboulanger@berklee.edu

Abstract. This paper presents a real-time vision-to-music system designed to explore an accessible and interactive auditory experience for people with visual impairments. A webcam feeds a Python/OpenCV application that identifies up to five prominent colored regions as object candidates and extracts each one’s position, relative size, and dominant color. These features travel to Csound over Open Sound Control (OSC), where five continuously running synthesis instruments turn them into an evolving soundscape: horizontal position sets stereo panning; vertical position controls pitch, chord voicing, or rhythmic pattern depending on the voice; relative size shapes amplitude, filter cutoff, reverberation, and delay; and the scene’s dominant color selects one of eight musical modes shared by all five voices, so color becomes harmony. The soundscape continues to evolve through timed chord changes and slow modulation even when the visual scene remains still. The system demonstrates how computer vision and real-time synthesis in Csound can support a musical, nonverbal form of environmental interaction alongside existing assistive approaches.

Keywords: Computer Vision, Csound, Open Sound Control, Interactive Music, Environmental Interaction, Visual Sound Design, Python, OpenCV

1 Introduction

Assistive technology for visual impairment often works through speech synthesis or warning tones in applications such as navigation, obstacle detection, and object recognition. Music offers a different channel: pitch, timbre, spatial position, rhythm, and amplitude can represent changes in what a camera sees while also producing an expressive listening experience. In this system, color is treated as a scene-level musical control, not a verbal label: after the dominant color satisfies the stability requirement, it selects the shared musical mode and chord progression. For example, a stable shift from a predominantly red scene to a predominantly blue scene changes the harmony from Mixolydian to Aeolian, while detected objects continue to influence panning, register, rhythm, loudness, and brightness through their position and relative size. This approach is intended to complement rather than replace existing accessibility tools.

Several sensory-substitution systems work in this space. *The vOICE* maps images onto acoustic properties [9]; *EyeMusic* assigns musical notes and instrument timbres to object position, shape, and color [10]; *SoundSight* uses smartphone sensor data, including color and spatial information, to generate real-time soundscapes [11]. Camera and gesture tracking have also driven creative performance directly: Pauline Oliveros’s Adaptive Use Musical Instrument (AUMI) turns tracked body movement into instrument control [12], while Boulanger’s work uses an infrared head-mounted gesture controller to enable expressive music performance for a user with quadriplegia [13]. Csound already supports this kind of work through image-processing opcodes and OSC [16,6], and Iain McCurdy’s *Csound Journal* columns turn visual phenomena into procedural synthesis [14,15], but Csound itself has no built-in object detection or recognition. Where *EyeMusic* and *SoundSight* assign discrete notes or timbres to visual features [10,11], the system described here runs five continuous synthesis processes in parallel, letting detected-object parameters modulate ongoing musical voices instead of triggering isolated events.

This project builds on that work rather than proposing a new paradigm: it converts visual information into real-time procedural sound. Python handles detection and color analysis using OpenCV [7,8]; Csound handles synthesis and spatial audio [1,2], so each tool does the job it is best suited for. A Python module extracts normalized horizontal and vertical position, relative size, and dominant color for the five largest detected object candidates and streams them to Csound over OSC, where

five continuously running instruments turn that stream into an evolving soundscape shaped by position, size, and a shared color-derived harmony. Two further mechanisms, detailed in Section 3, keep the texture moving when the scene itself is not changing, and vertical position maps to pitch, chord voicing, or rhythm depending on the voice, instead of following one rule across all five.

The contributions of this work are:

- A real-time vision-to-music interface designed for accessible and creative musical interaction.
- A modular Python/OSC/Csound architecture in which five continuously running synthesis voices can respond to the five largest detected object candidates for a layered, polyphonic response.
- A color-to-harmony mapping across eight musical modes, shared globally so the five voices remain harmonically coherent.
- An explicit account of the autonomous chord-progression and modulation layer that runs independently of the camera, so the camera’s actual contribution to the soundscape is stated rather than assumed.

2 System Architecture

The system has three independent parts: a Python/OpenCV computer vision module, an OSC communication layer, and a Csound synthesis engine [1,2], kept separate so the vision side and the audio side can each change without touching the other. The vision module converts each frame from BGR to HSV and constructs eight color masks. A pixel counts as chromatic when its saturation and value are both at least 45 out of 255; within that range, hue selects one of six bins: red ($\text{hue} \leq 7$ or ≥ 173), orange (8–19), yellow (20–34), green (35–84), blue (85–139), or purple (140–172). Two further classes sit outside the hue test: white (saturation below 35, value at least 180) and black (value at or below 50). An object’s dominant color is whichever class covers the most pixels inside its contour. Contours between roughly 1,500 pixels and 85% of the frame area, with a bounding box at least 30 pixels on each side, are kept as valid object candidates, which filters out small color-noise fragments as well as near-full-frame false positives caused by lighting changes [7,8]. Candidates are then ranked by contour area and truncated to the five largest, so when more than five objects are visible the module consistently favors the five largest candidates by contour area. For each of these objects the module computes a normalized horizontal position, vertical position, and relative size, then extracts its dominant color. Relative size is estimated from contour area and is used as a proximity-like control parameter rather than a physical distance measurement. Object identifiers 0–4 persist across frames through nearest-centroid tracking, instead of being reassigned from area rank on every frame. A separate pass computes the overall dominant color from the dominant-color pixel counts contributed by the detected objects. A color must account for at least 50% of the accumulated dominant-color pixels and remain stable for 2 seconds before it changes the musical mode, reducing rapid changes caused by camera noise or momentary occlusion. By default the two processes communicate over UDP on the loopback interface at port 7770 using the `python-osc` library, so running the system end to end needs no network configuration beyond a webcam, Python 3 with OpenCV, and Csound 6.18 on a single machine.

Each frame’s five largest candidates are matched to existing tracks by nearest centroid. The closest unmatched detection-track pair within a 120-pixel radius is assigned first, then the next-closest pair among what remains, until no pair falls within range; a detection left over claims a free slot if one of the five is open, and a track left over is simply held rather than reassigned by size rank. A physical object therefore stays bound to the same voice even as its size rank changes from frame to frame. If a tracked object goes unmatched for more than 0.5 seconds, it is treated as having left the frame: an explicit `/gone` message tells Csound to release that voice’s presence gate, instead of holding its last position and size indefinitely. Before that timeout its most recent values are simply held, so a lost track fades rather than cutting off abruptly.

Each `/object` message carries an object identifier, normalized horizontal and vertical position, relative size, and a color-class index as its OSC arguments; `/mode` carries a single color-class index for the confirmed scene color; and `/gone` carries the identifier of a track that has timed out. For example, an object at normalized position (0.72, 0.35) with relative size 0.41, classified as blue, and assigned identifier 2 arrives as `/object 2 0.72 0.35 0.41 4`; the trailing integer is the blue color-class index. A single receiver instrument listens for all three message types and writes them into indexed global control-rate variables that each synthesis instrument reads directly. All five synthesis instruments run continuously instead of firing discrete notes, reading these variables through `portk` smoothing,

which reduces jumps caused by frame-to-frame variations in the detected parameters [1,2,4]. Before any color has stabilized, pitch and chord degree fall back to the same Major Pentatonic tables used once white becomes the confirmed mode: `giMajorPent` for pitch, and `giProgMajorPent`, stepping through degrees 0, 3, 0, 4, for the chord progression. The orchestra defines seven instruments in total: the OSC receiver (instrument 1), the five synthesis voices (instruments 10 through 14), and a shared reverb/delay bus (instrument 90). The complete communication flow, from the Python vision module’s OSC messages through Csound’s receiver and into the synthesis instruments, is illustrated with implementation excerpts in Fig. 1. The full Python and Csound source, plus a demonstration video, is available in our GitHub repository [17].

```
# Python: OSC send                                ; Csound: OSC receive                                ; Csound: Instrument 10
...                                                ...                                                instr 10
client.send_message(                               kTrig OSClisten giHandle,
  "/mode",                                         "/object", "fffff",
  color_ids[active_color])                        kID, kX, kY,
...                                                kDistance, kColor
client.send_message(                               kModeTrig OSClisten
  "/object",                                     giHandle, "/mode",
  [float(obj["object_id"]),                       "i", kMode
   float(obj["x_norm"]),
   float(obj["y_norm"]),
   float(obj["distance"]),
   float(obj["color_id"])]])                     kGoneTrig OSClisten
...                                                giHandle, "/gone",
if elapsed >= track_timeout:                       "f", kGoneID
  client.send_message(
    "/gone", float(tid))
...
```

Fig. 1. Implementation excerpts showing the overall OSC communication flow between the Python vision module and Csound synthesis engine: Python sends `/object`, `/mode`, and `/gone` messages, Csound’s receiver instrument listens for all three, and a synthesis instrument reads the resulting global variables through `portk` smoothing. Ellipses indicate omitted code.

3 Sound Design and Interaction Mapping

The five synthesis voices, named as they appear in the Csound source, are the Harmonic Pad, Reverse FX, Bass Drone, FM Texture Lead, and Harmonic Bell. The Harmonic Pad layers five `oscili` voices through a `moogladder` low-pass filter, detuned not by fixed offsets but by four independent low-frequency oscillators drifting at roughly 0.035 to 0.09 Hz, for a warm, slowly shifting ensemble tone; Reverse FX produces short, step-sequenced melodic accents with an additional filtered-noise layer; the Bass Drone stacks a sub-octave, root, fifth, and octave through `oscili` and its own `moogladder` stage to reinforce the low end; the FM Texture Lead modulates one `oscili` carrier with another for evolving harmonic complexity through frequency modulation; and Harmonic Bell combines pitched oscillators with a filtered noise strike, again through `butlp` and `moogladder`, for percussive definition [1,2,3,4]. Each instrument reads one of the five tracked object slots, so up to five detected objects can drive five voices simultaneously instead of one shared position steering the entire ensemble. Each voice’s register is also fixed independently of any mapping: the Bass Drone anchors at MIDI note 36 (C2), the Harmonic Pad at 48 (C3), and Reverse FX and Harmonic Bell both at 72 (C5), so the ensemble stays spread across the range even as pitches move with the shared chord progression. The one parameter shared across all five voices is the color-derived mode. Once a color has stabilized, the active mode determines the scale material used throughout the system. Four voices also follow the corresponding mode-specific chord progression, while the FM Texture Lead quantizes its vertical-position-derived pitch directly to the active mode.

Horizontal position pans every voice, though the strength of the spatial cue varies: a slow panning LFO is layered in on most instruments, and each instrument sends a different proportion of its signal to a shared, cross-feeding reverb and delay bus, so drier voices keep a clearer pan and wetter voices lose some of it into the bus. That bus uses `reverbsc` with a feedback level of 0.90 and a 10 kHz cutoff, together with cross-fed delays of 0.32 and 0.46 seconds. Vertical position’s effect depends on the voice rather than one uniform mapping: in the FM Texture Lead it is quantized directly to pitch through the `QuantFreq` user-defined opcode (UDO), using scale degrees from the active mode; in the Harmonic Pad it instead selects among eight chord voicings, changing registral spread rather than pitch; in Reverse FX and Harmonic Bell it selects among four rhythmic patterns; and the Bass Drone ignores it, taking its root pitch entirely from the shared chord progression described

below [1,2,3,4,5]. In Reverse FX, size sets the step sequencer’s rate directly, from 1.0 up to about 3.8 steps per second, so a larger object drives a faster, busier arpeggio through an eight-step pattern built from the active chord’s root, third, fifth, seventh, and ninth. Harmonic Bell’s own step rate rises with size the same way, from 2.8 up to 6.0 steps per second, making it the faster and more percussive of the two rhythmic voices at close range. Relative object size drives amplitude, low-pass filter cutoff, reverberation, and delay in all five voices. The **DistanceFX** opcode maps this size-derived value inversely for effects control, so larger objects generally sound brighter and more direct, while smaller ones receive more ambience. In the FM Texture Lead specifically, size does more than this: it directly scales the frequency-modulation depth from 35 up to roughly 155, so a larger object increases the spectral complexity of the FM voice, not simply its loudness or brightness. Fig. 2 shows the **DistanceFX** mapping and the corresponding parameter mappings used in the FM Texture Lead. Dominant color sets the shared mode across eight pairings, each fixing its own four-step chord-degree progression through **ModeChordDegree**. Phrygian and Locrian happen to share the same progression despite belonging to different modes, since the shared timer described below still walks all four voices through the same scale-degree sequence regardless of which of the two is active.

<pre> ; Csound: DistanceFX UDO opcode DistanceFX, k, k kDistance xin kDistance limit kDistance, 0, 1 kMapped = limit((kDistance - 0.25) / 0.50, 0, 1) kFXAmount = 1 - kMapped xout kFXAmount endop </pre>	<pre> ; Csound: Instrument 13 - Mapping ... kX portk gkX3, 0.08 kY portk gkY3, 0.25 kD portk gkD3, 0.10 kA portk gkA3, 0.25 kRoot QuantFreq kY, 1 kAmp = kA * (0.024 + kD * 0.10) kModDepth = 35 + (kD * 120) ... </pre>	<pre> ; Csound: Instrument 13 - Synthesis ... aMod oscili kModDepth, kRoot * 0.35 aCar poscil kAmp, kRoot + aMod aLayer1 poscil kAmp * 0.45, kRoot * 2 aLayer2 poscil kAmp * 0.30, kRoot * 0.5 ... </pre>
--	---	--

Fig. 2. Exact Csound excerpts mapping visual parameters to synthesis controls: **DistanceFX** maps object size inversely to effects amount, while Instrument 13 uses smoothed parameters for quantized pitch, amplitude, and FM depth. Ellipses indicate omitted code.

Two mechanisms operate independently of the camera. An internal timer, measured from the most recent color-mode change rather than from system startup, steps a four-chord progression every 5 seconds in four of the five voices, the Harmonic Pad, Reverse FX, Bass Drone, and Harmonic Bell, returning to the tonic every 20 seconds; changing the dominant color resets this cycle to its first chord, while simply holding a color lets it keep looping regardless of what else happens in view. The FM Texture Lead is exempt. Separately, a set of slow, free-running low-frequency oscillators across the five voices adds gentle amplitude drift, detuning, and, in four of the five voices, pan drift, keeping the texture moving between camera updates or when detected objects are stationary, since the camera does not drive all of the audible motion. Mode changes are intentionally limited by the 2-second stability requirement, and **portk** smoothing further reduces abrupt changes. The system uses one dominant color for the whole scene rather than a separate harmonic mode per object, so all five voices share a single harmonic identity even when the detected objects contain different colors. Each frame ranks candidates by size and truncates them to the five largest before matching begins, so only those five are considered for tracking; an existing track can still hold its slot through the 0.5-second timeout even while a larger, untracked object appears elsewhere in the frame.

4 Conclusion and Future Directions

Before any broader release, we plan to profile end-to-end latency and frame-rate stability under sustained multi-object tracking. Future work will extend the color-to-harmony mapping toward instrument selection and spectral character, add further synthesis voices and UDOs within the same modular design [1,2,3,4,5], and weigh a native Swift/iOS and Android implementation against a browser-based alternative using Csound WebAssembly with a browser-native machine learning framework such as ml5.js or TensorFlow.js for zero-install, cross-platform access. We also plan user studies with visually impaired participants and intend to involve them at the design stage itself rather than only in evaluation. Beyond assistive use, the same modular framework may also support creative and educational vision-to-music applications.

References

1. Lazzarini, V., Yi, S., ffitch, J., Heintz, J., Brandtsegg, Ø., McCurdy, I.: *Csound: A Sound and Music Computing System*. Springer, Cham (2016)
2. Boulanger, R. (ed.): *The Csound Book: Perspectives in Software Synthesis, Sound Design, Signal Processing, and Programming*. MIT Press, Cambridge (2000)
3. Boulanger, R., Lazzarini, V. (eds.): *The Audio Programming Book*. MIT Press, Cambridge (2010)
4. FLOSS Manuals Foundation: *The Csound FLOSS Manual*. https://www.csound-tutorial.net/floss_manual/
5. CsoundQt Developers: *CsoundQt Examples Collection*. <https://github.com/CsoundQt/CsoundQt/tree/master/src/Examples>
6. Wright, M., Freed, A.: Open Sound Control: A New Protocol for Communicating with Sound Synthesizers. *Proc. International Computer Music Conference* (1997)
7. OpenCV Developers: *OpenCV Documentation*. <https://docs.opencv.org/4.x/>
8. Kaehler, A., Bradski, G.: *Learning OpenCV 3: Computer Vision in C++ with the OpenCV Library*. O'Reilly Media (2016)
9. Meijer, P. B. L.: An Experimental System for Auditory Image Representations. *IEEE Trans. Biomedical Engineering*, 39(2), 112–121 (1992)
10. Abboud, S., Hanassy, S., Levy-Tzedek, S., Maidenbaum, S., Amedi, A.: EyeMusic: Introducing a “Visual” Colorful Experience for the Blind Using Auditory Sensory Substitution. *Restorative Neurology and Neuroscience*, 32(2), 247–257 (2014)
11. Hamilton-Fletcher, G., Alvarez, J., Obrist, M., Ward, J.: SoundSight: A Mobile Sensory Substitution Device That Sonifies Colour, Distance, and Temperature. *Journal on Multimodal User Interfaces*, 16(1), 107–123 (2022)
12. Oliveros, P., Miller, L., Heyen, J., Siddall, G., Hazard, S.: A Musical Improvisation Interface for People with Severe Physical Disabilities. *Music and Medicine*, 3(3), 172–181 (2011)
13. Boulanger, A.: Expressive Gesture Controller for an Individual with Quadriplegia. *Proc. 10th ACM Conference on Ubiquitous Computing (UbiComp 2008)*, 113–116 (2008)
14. McCurdy, I.: *Interpreting Visual Phenomena using Csound – Part 1: Futile Movement*. Csound Journal, Issue 21 (2015)
15. McCurdy, I.: *Interpreting Visual Phenomena using Csound – Part 2: The Stroboscopic Effect and Pendulum Waves*. Csound Journal, Issue 22 (2016)
16. Csound Developers: *Image Processing Opcodes*. The Canonical Csound Reference Manual. <https://csound.com/docs/manual/ImageopcodesTop.html>
17. Chen, X., Boulanger, R.: *Vision-to-Music System: Source Code and Demonstration Assets*. GitHub repository. <https://github.com/WillChen5258/Vision-To-Music-System>